

SmartDoc AI - Intelligent Document QA System

Bùi Nguyên Thịnh*, Nguyễn Tuấn Tài[†], Nguyễn Văn Quốc Huynh[‡], Phạm Yến Nhi[‡], Kiều Hoài Nam[‡]

[‡]Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn

Email: tbinguyen.tb@gmail.com, tuantai1915@gmail.com, quochuynh626@gmail.com, pham nhi839292@gmail.com, namhoaikieu009@gmail.com

Abstract—Cùng với sự gia tăng mạnh mẽ về lượng tài liệu học thuật và văn bản số, việc tìm kiếm và trích xuất thông tin một cách thủ công đang dần trở nên mất thời gian và kém hiệu quả. Mặc dù các mô hình ngôn ngữ lớn (LLM) thương mại như ChatGPT giải quyết tốt bài toán tổng hợp tri thức, chúng vẫn đối mặt với hai rào cản chí mạng khi áp dụng vào cấp độ doanh nghiệp: rủi ro an toàn thông tin (Privacy) khi đưa dữ liệu nội bộ lên đám mây, và nội dung ảo giác (Hallucination) khi AI bịa đặt thông tin. Báo cáo này trình bày thiết kế chi tiết và tiến trình triển khai phần mềm mã nguồn mở SmartDoc AI, một trợ lý phân tích tài liệu (PDF, DOCX) chạy độc lập (100% Offline Local). Xoay quanh nền tảng kiến trúc Retrieval-Augmented Generation (RAG), hệ thống kết hợp mô hình Qwen mở thông qua nền tảng suy luận Ollama cùng framework LangChain và Streamlit. Báo cáo cung cấp góc nhìn sâu sắc về nền tảng toán học của thuật toán truy xuất, phân rã mã nguồn thành các luồng xử lý riêng biệt. Đi kèm với sự phối hợp các kỹ thuật tiên tiến: Hybrid Search (BM25 + FAISS), Mô hình chấp Cross-Encoder Re-ranking chống nhiễu, luồng thuật toán đệ quy Co-RAG (Chain of Retrieval) tự kích hoạt truy xuất bổ sung và Self-RAG. Hệ thống cũng giải quyết triệt để rào cản về kiểm soát đa tài liệu bằng kiến trúc Singleton Caching và quản trị JSON State.

Index Terms—RAG; LLM; FAISS; Open Source Architecture; Qwen; Hybrid Multi-Search; BM25; Self-RAG; Streamlit.

I. GIỚI THIỆU CHUNG VÀ BỐI CẢNH

Trong nhiều môi trường học thuật và doanh nghiệp, tri thức thường nằm rải rác trong tài liệu PDF/DOCX. Khi số lượng tài liệu tăng, việc tra cứu thủ công trở nên chậm, khó nhất quán và khó truy vết nguồn. Bên cạnh đó, việc đưa dữ liệu nội bộ lên các dịch vụ AI bên ngoài có thể phát sinh rủi ro về bảo mật và tuân thủ.

Để giải quyết bài toán này, hướng tiếp cận **Retrieval-Augmented Generation (RAG)** được lựa chọn: hệ thống chỉ tạo câu trả lời sau khi truy xuất

được ngữ cảnh liên quan từ kho tài liệu cục bộ. Cách làm này giúp giảm nguy cơ trả lời ngoài phạm vi dữ liệu và tăng khả năng kiểm chứng thông tin qua nguồn trích dẫn.

Từ bối cảnh trên, dự án **SmartDoc AI** được phát triển với mục tiêu xây dựng một trợ lý hỏi đáp tài liệu chạy **offline/local-first** dựa trên mã nguồn mở. Hệ thống hỗ trợ nạp nhiều tài liệu PDF/DOCX, tạo chỉ mục FAISS từ embedding ngôn ngữ, truy xuất lai (vector + BM25), tái xếp hạng bằng Cross-Encoder và phản hồi qua giao diện Streamlit. Ngoài chức năng hỏi đáp cơ bản, dự án còn tích hợp truy vấn theo ngữ cảnh hội thoại, bộ lọc metadata đa tài liệu và cơ chế fallback mô hình khi tài nguyên hạn chế.

II. NỀN TẢNG CƠ SỞ KIẾN TRÚC ĐA MÔ HÌNH (MULTI-MODEL ARCHITECTURE)

SmartDoc AI được tổ chức theo kiến trúc truy xuất lai gồm nhiều thành phần phối hợp: mô hình embedding để biểu diễn ngữ nghĩa, FAISS cho truy xuất vector, BM25 cho truy xuất từ khóa và Cross-Encoder cho bước tái xếp hạng. Cách thiết kế này giúp hệ thống cân bằng giữa độ bao phủ thông tin và độ chính xác ngữ cảnh trước khi chuyển dữ liệu vào mô hình sinh đáp án.

A. Cơ Chế Nhúng Không Gian (Dense Embedding Mechanism)

Mỗi đoạn văn bản (text chunk) được ánh xạ sang vector embedding bằng mô hình sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2. Khi người dùng đưa ra truy vấn, FAISS thực hiện tìm kiếm theo độ tương đồng ngữ nghĩa để lấy ra các đoạn liên quan nhất. Tập ứng viên này đóng vai trò ngữ cảnh nền cho bước suy luận tiếp theo.

B. Cơ Chế Khớp Ký Tự Từ Vựng (Sparse BM25 Mechanism)

Lỗi hỏng của quy trình Vector RAG thông thường nằm ở loại "Cụm danh từ cụ thể". Khi lượng thông tin bao gồm mã số, hệ Vector lưới thường bỏ qua do thiếu vắng kết nối ngữ nghĩa tính. Bù khuyết lỗi lầm, SmartDoc đưa vào vận hành cơ chế BM25. Thuật toán BM25 gán trọng số tự động cho từng từ khóa riêng lẻ từ truy vấn, triệt tiêu sức nặng của các từ nổi vô nghĩa. Sự kết hợp sức đồng thời giữa tuyến BM25 (Ngữ vựng phân mảnh) và FAISS (Ngữ nghĩa) hình thành bộ máy dò tìm diện rộng vô cùng chuẩn xác.

C. Hệ Thống Tái Đánh Giá Chuyên Sâu (Cross-Encoder Re-Ranking)

Tập danh sách bối cảnh trào ngược ra từ hệ tìm kiếm chéo dễ bị dính lẫn lộn "Văn bản rác". Quần thể này bị ném qua mạng Nơ-ron bộ lọc siêu việt ms-marco-MiniLM-L-6-v2. Mô hình Cross-Encoder nhận ma trận chéo bao gồm cặp Câu Hỏi Người Dùng và Bối Cảnh, sau đó ép vào tầng chập Transformer sinh ra xác suất tương quan (Logit Score). Các đoạn những có hệ số phân giải tỷ lệ dưới ngưỡng quy định sẽ bị loại trừ khắc nghiệt, đảm bảo độ tinh khiết tối đa của dữ liệu trước khi bơm vào hệ não bộ LLM khởi tạo câu trả lời.

III. KIẾN TRÚC MÁY TRẠNG THÁI VÀ LUỒNG GỌI HÀM

SmartDoc AI tổ chức luồng xử lý theo ba lớp: Presentation (Streamlit), Application (business logic), và Data Layer (storage/retrieval). Luồng từ UI xuống phía dưới tuân thủ nguyên tắc gọi hàm đồng bộ, với trạng thái phiên được lưu trữ qua Streamlit `session_state` và JSON persistence.

A. Quy trình nạp tài liệu (Ingestion Pipeline)

Quy trình nạp từng file tuân thủ bốn bước:

- 1) Ghi file vào `data/raw` thông qua `save_uploaded_file`.
- 2) Tách chunk bằng `RecursiveCharacterTextSplitter` theo cấu hình kích cỡ.
- 3) Bổ sung metadata (`source`, `file_type`, `upload_time`, `upload_date`) cho từng đoạn thông qua `enrich_chunks_metadata`.
- 4) Tạo hoặc cập nhật index FAISS bằng `build_and_save_faiss_index` hoặc `update_faiss_index`.

Các file được xử lý tuần tự.

B. Tối ưu hóa tài nguyên (Resource Caching)

Streamlit gọi lại mã script sau mỗi tương tác người dùng, điều này có thể dẫn đến nạp lại các mô hình đắt đỏ. Để giảm chi phí, hệ thống dùng `@st.cache_resource` để cache embedding model và cross-encoder model. Nhờ vậy, các mô hình chỉ được tải lần đầu tiên, cải thiện đáng kể hiệu năng và giảm sử dụng bộ nhớ trong các phiên làm việc kéo dài.

C. Kỹ Thuật In Vết Lịch Sử Cấp Phát Ngữ Nghĩa (Metadata)

Các chunk embedding sẽ giảm giá trị sử dụng nếu thiếu thông tin truy vết nguồn. Vì vậy, hàm `enrich_chunks_metadata` bổ sung metadata như `source`, `file_name`, `file_type`, `upload_time` và `upload_date` cho từng đoạn văn bản. Nhờ đó, quá trình retrieval có thể lọc theo điều kiện metadata và hiển thị trích dẫn theo đúng tài liệu gốc.

IV. ĐỒ THỊ KHAI PHÁ TẬP VÔ CẤU TRÚC (PARSING)

Tầng Parsing của SmartDoc AI tập trung vào mục tiêu chuẩn hóa dữ liệu đầu vào không cấu trúc (PDF, DOCX) thành văn bản để đưa vào pipeline chunking và indexing. Thiết kế hiện tại ưu tiên tính ổn định và khả năng mở rộng hơn là các bước phân tích cấu trúc tài liệu chuyên sâu.

A. Phân Tách Bề Mặt Mảng PDF (PDFPlumber-Loader)

Với tệp PDF, hệ thống sử dụng `PDFPlumber-Loader` để trích xuất nội dung văn bản theo trang và trả về danh sách tài liệu cho các bước xử lý tiếp theo. Cách làm này phù hợp cho truy xuất ngữ nghĩa tổng quát, đồng thời giảm độ phức tạp triển khai khi cần nạp nhiều loại tài liệu trong cùng một luồng ingest.

B. Bóc Tách Cây Phả Hệ Word (Python-Docx Tree Mapper)

Với tệp DOCX, hệ thống dùng `python-docx` để đọc các đoạn văn bản (paragraph), loại bỏ dòng rỗng và ghép thành chuỗi nội dung thống nhất. Sau bước trích xuất, dữ liệu tiếp tục đi qua chunking và được bổ sung metadata để phục vụ lọc theo nguồn tài liệu trong giai đoạn retrieval.

V. KIẾN TRÚC LUỒNG HỎI ĐÁP VÀ TÁI TỔ HỢP ĐỆ QUY

Quy trình Hỏi Đáp không dừng lại ở Fetch dữ liệu mà bắt đầu kích hoạt cơ chế sinh học suy diễn. Luồng tương tác mang định dạng máy dò tìm AI tự chủ.

A. Luồng Gọi Mảng Hybrid Chống Khuyết Thiếu

Algorithm 1 Cơ chế Truy xuất Bối cảnh Tri Thức Đa Tầng Hybrid Auto-Retrieval Architecture

- 1: Trạm nhận câu truy vấn thô Q do User nhập thông qua Sidebar.
- 2: **if** `is_follow_up_query(Q)` == True logic **then**
- 3: Mở khóa Bộ Nhớ **Session JSON Memory**.
- 4: Cập nhật LLM viết lại Q thành câu **StandardAlone Query**.
- 5: **end if**
- 6: Kích hoạt Regex tìm Cờ Đích Danh từ mảng văn bản.
- 7: Chạy **Nhánh Trái**: Lọc K đoạn Top K bằng FAISS Vector.
- 8: Chạy **Nhánh Phải**: Lọc N đoạn Top N bằng Sparse BM25 Keyword TF-IDF.
- 9: Gom Set vào mảng tinh sạch Thuật toán Giao thoa.
- 10: Giao List cho module **CrossEncoder Re-ranker**:
- 11: **for** Mỗi nút đoạn V trong tập **do**
- 12: Tính điểm liên quan giữa truy vấn và đoạn văn bản.
- 13: **end for**
- 14: Sắp xếp điểm theo thứ tự giảm dần và chọn Top-K đoạn phù hợp nhất.
- 15: Nạp Mảng Trượt **Ngữ Cảnh** tinh chế lên Streamlit.

Thuật toán truy xuất tại Algorithm 1 tích hợp cơ chế lọc theo nguồn tài liệu dựa trên metadata, giúp thu hẹp không gian tìm kiếm khi truy vấn nêu rõ tên tài liệu. Cách tiếp cận này thường cải thiện độ tập trung của ngữ cảnh trả về; mức cải thiện tốc độ cụ thể phụ thuộc dữ liệu và cấu hình thực nghiệm.

B. Quản Trị Vòng Đời Data (Clear Vector Data)

Tính năng dọn dữ liệu tạm được kích hoạt bởi hàm `clear_vector_store_data`. Hàm này xóa dữ liệu trong `data/index` và `data/raw`, đồng thời giao diện reset các trạng thái liên quan đến index/retrieval để bắt đầu phiên làm việc mới. Cơ chế này giúp hạn chế tồn đọng dữ liệu cũ và giảm rủi ro nhiễu ngữ cảnh khi ingest lại tài liệu.

C. Tiến Trình Đánh Giá Củng Cố Đệ Quy Tiên Tiến (Recursive Co-RAG Cognitive Process)

RAG tính vòng lặp lỗi thời lộ rõ giới hạn trước các câu hỏi khuyết nhịp nối. Giải pháp vĩ mô `corag_chain_manager.py` vận hành một thiết bị sinh học giả nguyên. Module cấp cho Agent quyền hạn tự rà soát.

Algorithm 2 Phân Tích Logic Vòng lặp Đệ quy Co-RAG Assessment Agent

- 1: Xây vùng nhớ chứa **Bối cảnh tập con C**.
- 2: **for** Khởi chạy $i = 1$ tới mốc khóa cứng **Max_Rounds** **do**
- 3: Chuyển giao Flag cho hệ Cổ Máy Thẩm Phán **Judge-LLM** rà soát:
"Bối cảnh C thu được đã đủ giải đáp Truy Vấn Mẹ chưa?"
- 4: **if** `Judge_Result` nhả ra mã **SUFFICIENT** **then**
- 5: Ép ngắt đột ngột (**BREAK**), đẩy C ra Engine Text Generation.
- 6: **else**
- 7: Phân tách List thông tin bị Thiếu Hụt bù trừ.
- 8: Nhập mảng lập thể: `Sub_Query`.
- 9: Thổi còi Khởi động Tìm kiếm Nhúng mới với mã ngõ vào móc nối `Sub_Query`.
- 10: Đắp trộn Mảnh C gốc + Dữ kiện Mới mẻ vớt được ở vòng ngầm.
- 11: Loại bỏ các đoạn trùng lặp theo nội dung đã tích lũy trong ngữ cảnh.
- 12: **end if**
- 13: **end for**
- 14: Khớp Cửa sổ băng thông Token và in Kết Quả Render lưới giao diện GUI.

Co-RAG (Algorithm 2) nhồi vào hệ tính RAG một tri giác phán đoán (Agentic Flow). Khả năng tự lằm nhảm: "Ngữ nghĩa thiếu về B, phải gọi lệnh `Sub_Query` quét thêm File Phụ tấp dệm củng cố".

VI. TỐI ƯU KỸ THUẬT VI MÔ CHỊU LỖI

Một cỗ máy tinh xảo cần vỏ giáp bảo hộ (Fault-Tolerance).

A. Cơ Chế Kích Cầu Lỗi Đệm OOM Tình (Out-of-Memory Fallback Tolerance)

Tầng suy luận triển khai cơ chế **retry + fallback** trong module `ollama_inference_engine`. Khi lời gọi `invoke` gặp nhóm lỗi có thể thử lại (ví dụ: thiếu bộ nhớ, lỗi kết nối, timeout, hoặc phản hồi

5xx), hệ thống đánh dấu model hiện tại là không ổn định trong phiên và lần lượt thử các model dự phòng được cấu hình trong FALLBACK_MODELS (như qwen2.5:1.5b, qwen2.5:0.5b). Cơ chế này giúp giảm xác suất gián đoạn truy vấn khi backend Ollama gặp lỗi tạm thời; tuy nhiên luồng gọi hiện tại là đồng bộ theo cơ chế invoke, không phải streaming token liên tục.

B. Mũi Khoan Lỗ Khóa An Toàn Tác Dụng Ngược (Restrictive Prompt Layer)

LLM cấu trúc mở tự do Qwen Model rất dễ "Ảo giác mạng lân cận". System Instruction nhồi khối tĩnh thép rắn đe: *"ONLY meticulously use the Provided Base Embedded Contexts. If NOT match semantic string, explicitly DECLARE 'I Cannot Provide External Out-of-Base Information'. Do NOT Hallucinate!"*. Vành đai lệnh chặn này khóa chết rủi ro cung cấp tư liệu mật giả, đúc AI thành tay quản thư bảo thủ và chính trực dưới bối cảnh Local Offline.

VII. KHẢO SÁT THỰC NGHIỆM ĐỊNH LƯỢNG

Trong phạm vi thời gian thực hiện đồ án, nhóm thực hiện một đợt khảo sát định lượng sơ bộ nhằm so sánh ba cấu hình chunk của Basic RAG: **500/50**, **1000/100** và **2000/200**. Mỗi cấu hình được kiểm tra trên cùng một bộ **3 truy vấn tóm tắt** để bảo đảm tính nhất quán về bối cảnh đánh giá.

Với mỗi truy vấn, nhóm ghi nhận: (i) thời gian phản hồi, (ii) độ tin cậy chủ quan theo thang 1–10, và (iii) điểm chất lượng theo thang 0–2. Điểm chất lượng được chấm như sau:

- **0 điểm:** trả lời sai, hoặc không có thông tin trong tài liệu nhưng vẫn suy diễn.
- **1 điểm:** trả lời đúng một phần, thiếu ý chính hoặc nhầm lẫn sự kiện.
- **2 điểm:** trả lời đúng trọng tâm, đủ ý chính, bám sát nội dung tài liệu.

Điểm chất lượng trung bình được quy đổi theo các mức diễn giải sau:

- **Tốt** khi điểm trung bình từ 1.6 đến 2.0.
- **Trung bình** khi điểm trung bình từ 1.2 đến dưới 1.6.
- **Thấp** khi điểm trung bình dưới 1.2.

Bảng I phản ánh kết quả đo thực tế từ bộ kiểm thử 3 truy vấn trên từng cấu hình chunk. Do cỡ mẫu còn nhỏ, kết quả hiện tại mang ý nghĩa tham chiếu ban đầu để định hướng các vòng đo mở rộng.

Hình 1 cho thấy cấu hình **500/50** đạt điểm chất lượng trung bình cao nhất (1.67), tiếp đến là **1000/100** (1.33), và **2000/200** thấp nhất (1.00). Kết quả này

Table I
BẢNG 1. KẾT QUẢ KHẢO SÁT ĐỊNH LƯỢNG THEO CẤU HÌNH CHUNK (3 TRUY VẤN KIỂM THỬ)

Cấu hình chunk	Độ trễ TB (s)	Điểm chất lượng TB (0–2)
500/50	169.3	1.67
1000/100	177.3	1.33
2000/200	159.3	1.00

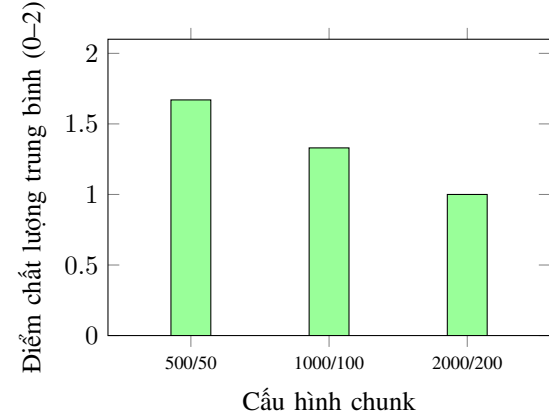


Figure 1. Điểm chất lượng trung bình theo từng cấu hình chunk (trích từ Bảng I).

nhất quán với quan sát định tính trong quá trình chấm từng câu: cấu hình 500/50 trả lời đúng trọng tâm nhiều hơn, trong khi 2000/200 thường chỉ đạt mức đúng một phần ở các ý then chốt. Ở giai đoạn tiếp theo, nhóm sẽ mở rộng số lượng câu kiểm thử để tăng độ tin cậy thống kê trước khi kết luận cấu hình tối ưu.

VIII. PHÂN CẤP QUẢN TRỊ BÀI TOÁN HỆ RAG KHÔNG NGOAN MỞ RỘNG

Nhằm đạt đỉnh cao tương tác liền mạch, dự án vượt rào mảng hỏi đáp Question & Answering một bước thô bạo. Sự lắp ráp hai modul bổ trợ này nổi rộng không gian thực phi tuyến:

A. Lưới Phễu Siêu Dữ Liệu Đa Ràng Buộc (Metadata Dynamic Filtering)

Mở rộng chức năng tải tập giấy tờ số lượng lớn (Multi-Document Ingestion) gây nên tai họa chồng chéo ngữ nghĩa đối lập. Nếu hệ tìm kiếm FAISS nhắm mắt quét ngang toàn bộ hệ sinh thái đồ thị, câu trả lời sẽ dễ lỏng ghép râu ông nọ cắm cằm bà kia của hai tệp văn bản khác nhau. Để phong tỏa, động cơ ném thêm bộ **Metadata Filter** vào ống kẹp của Retrieval. Mỗi file PDF đẩy vào hệ thống

không chỉ bóp vụn thành Vector, mà bị đóng một tem thẻ nhãn điện tử (Metadata Tags) mang tên: Tên nguồn (source), Định dạng (file_type), và Dấu hiệu thời gian upload_date. Khi mở rộng ingest đa tài liệu, nguy cơ trộn ngữ cảnh giữa các nguồn tăng đáng kể. Để giảm thiểu, hệ thống gắn metadata cho từng chunk (source, file_type, upload_date) ngay sau bước tách văn bản từ tài liệu PDF/DOCX. Trong giai đoạn retrieval, bộ lọc metadata được áp dụng đồng thời cho nhánh vector (MMR/similarity) và tập tài liệu đầu vào của BM25 trước khi hợp nhất kết quả. Cách làm này giúp thu hẹp không gian tìm kiếm theo phạm vi người dùng chọn, từ đó cải thiện tính liên quan của ngữ cảnh; tuy nhiên không phải cơ chế đảm bảo loại bỏ hoàn toàn hiện tượng hallucination.

B. Quản Lý Bộ Trạng Thái Thu Lịch Sử (Conversational Memory Cache)

Luồng truy xuất mặc định là stateless, vì vậy các câu follow-up thiếu chủ ngữ có thể làm giảm độ chính xác truy xuất nếu không bổ sung ngữ cảnh hội thoại. SmartDoc xử lý điểm này bằng cơ chế phát hiện follow-up, sau đó viết lại truy vấn dựa trên lịch sử gần nhất trong chat_history trước khi gửi xuống retriever. Trạng thái hội thoại được duy trì trong st.session_state và đồng bộ xuống JSON để phục hồi phiên làm việc. Nhờ đó, các câu hỏi nối tiếp có thể được chuyển về dạng truy vấn độc lập rõ nghĩa hơn, giúp pipeline RAG/Co-RAG chọn ngữ cảnh phù hợp hơn ở các vòng truy xuất tiếp theo.

IX. KẾT LUẬN MỀM KIẾN TRÚC VÀ ĐỊNH HƯỚNG TƯƠNG LAI VẬN CHUYỂN RAG TÍCH HỢP VĨ MÔ DOANH NGHIỆP CẤP NẶNG

SmartDoc AI theo định hướng local-first khi triển khai suy luận với Ollama trên máy cục bộ, nhờ đó giảm phụ thuộc vào dịch vụ bên ngoài và hỗ trợ tốt hơn cho bối cảnh dữ liệu nội bộ. Tuy nhiên, hệ thống không mặc định triệt tiêu toàn bộ rủi ro vận hành; mức an toàn thực tế vẫn phụ thuộc cấu hình hạ tầng, chính sách truy cập và quy trình quản trị dữ liệu của đơn vị triển khai. Ở góc độ kiến trúc, dự án tách rõ bốn lớp Presentation, Application, Data Layer và Model Layer. Trên nền đó, pipeline Hybrid Search kết hợp với bước Cross-Encoder re-ranking giúp tăng chất lượng ngữ cảnh trước khi sinh đáp án. Bên cạnh đó, cơ chế retryable-error + fallback model giúp giảm xác suất gián đoạn khi mô hình chính gặp lỗi tạm thời (ví dụ thiếu bộ nhớ hoặc lỗi

kết nối), nhưng không được xem là bảo đảm tuyệt đối trong mọi tải vận hành.

Tương lai hệ thống sẽ tháo gỡ điểm đen giới hạn đồ họa văn bản bằng kỹ nghệ máy nhìn quang học **Vision-Language Models (VLM Image Optical Machine Layer)**. Tích hợp bút phá RAG trôi lên bề mặt phân tích hình thù lưới bằng nhúng phẳng, mở ra thời đại kết xuất Microservice API qua Data Cloud vĩnh cửu.

THƯ MỤC TÀI LIỆU THAM KHẢO HỌC THUẬT

- [1] LangChain AI, *LangChain Document Core Framework Ecosystem Architecture Reference Guide*. LangChain Foundation System Index Documentation Organization, 2026. https://python.langchain.com/docs/core_architecture_ecosystems
- [2] Ollama Open Source, *Get Local Workload Setup Guide Application Protocol Running Secure Local Offline Native Inference*. Global Community Software Repository Platform, 2026. <https://ollama.com/library/docs/>
- [3] Alibaba Cloud Qwen AI, *Qwen Series Technical Protocol Analysis Report: Deep Dense Neural Model Learning*. Alibaba Cloud AI Artificial Research Institute, December 2024. arXiv Advanced Preprint.
- [4] Jeffrey Johnson, Matthijs Douze, and Hervé Jégou, *Billion-scale Heavy Matrix Array Dense Similarity Mathematical Search Architecture Vector Machine Data Point Logic*, IEEE Technical Heavy Index Neural Networking Database Analysis Metrics, vol. 7, issue 9, pp. 535-547, 2019.
- [5] Patrick Lewis, *Khái Niệm Lý Thuyết Thiết Kế Hệ Khung Kiến Trúc Retrieval-Augmented Retrieval System Engine*, Neural Information Processing Systems (NeurIPS 2020), 2020.
- [6] Streamlit Framework UI, *Rapid Streamlit Development Core Pipeline Ecosystem Python Build Data Science Rapid Generation*. https://docs.streamlit.io/library/core_api_rendering
- [7] T. Wolf, et al., *Kho Tàng Framework Core Component Machine Transformers Neural Complex Auto-Attention Block Processing Ecosystem Protocol*, EMNLP Mechanism Algorithm Group Deep Structure 2020. <https://huggingface.co/docs>
- [8] Stephen Robertson, *The Statistical Sparse Distribution Mathematics Model Core Keyword Search Term Evaluation Probabilistic Factor Algorithm Index Rank BM25*, Foundations of Information Engine Algorithms, 2009.